

به نام خدا



**Department of
Computer Engineering**

دانشکده مهندسی
کامپیوتر و فناوری اطلاعات

تمرین سوم

رایانش ابری

دکتر احمد جوادی

فاز اول: آشنایی با Hadoop

مقدمه

در فاز اول پروژه شما با ابزار Hadoop آشنا خواهید شد. Hadoop یک چارچوب متن‌باز برای پردازش حجم‌های بسیار بزرگ داده است که آن‌ها را بین چندین سرور پخش می‌کند و به‌صورت موازی پردازش انجام می‌دهد. Hadoop از دو بخش اصلی تشکیل شده: HDFS که یک سیستم فایل توزیع‌شده است و داده‌ها را روی چندین سرور با قابلیت تحمل خطا ذخیره می‌کند، و MapReduce که یک مدل پردازشی است و کار را به دو بخش Map و Reduce تقسیم می‌کند. نتیجه این است که می‌توان داده‌هایی را که روی یک کامپیوتر قابل پردازش نیستند، روی ده‌ها یا صدها ماشین پردازش کرد، بدون اینکه برنامه‌نویس درگیر جزئیات شبکه و توزیع کار شود.

بخش اول: اجرای کلاستر

در این بخش باید کلاستر Hadoop شامل یک namenode و یک datanode را اجرا کنید. برای بالا آوردن کلاستر فایل docker-compose در اختیار شما قرار گرفته، که می‌توانید با دستور docker compose up آن را بالا آورده و با دستور docker compose down آن را متوقف کنید.

بعد از اجرای کانتینرها می‌توانید در آدرس‌های زیر به ترتیب رابط گرافیکی اطلاعات مربوط به namenode و DataNode خود را مشاهده کنید:

127.0.0.1:9864

127.0.0.1:9870

با اجرای دستور `hdfs dfs -put` درون کانتینر NameNode، فایل داده `access-2019-01-23.log` را به DataNode منتقل کنید.

تسک ۱.۱: موارد زیر را به گزارش خود اضافه کنید:

- نمایش کانتینرهای ایجاد شده
- توضیح وظایف هر کدام از کانتینرها
- نمایش اطلاعات DataNode از طریق رابط گرافیکی (بعد از اضافه کردن فایل داده به datanode)

بخش دوم: اجرای MapReduce بر روی کلاستر

در این بخش باید با استفاده از Hadoop Streaming و دو اسکریپت پایتون (mapper و reducer)، یک برنامه MapReduce برای تحلیل لاگ‌های دسترسی وب‌سرور بنویسید.

خواندن Access Logs

در محصولات بزرگ، سرورهای وب مثل Apache یا Nginx هر روز میلیون‌ها درخواست را در `access log` خود ثبت می‌کنند، حجم این لاگ‌ها خیلی زود به اندازه‌ای می‌رسد که یک ماشین به‌تنهایی نمی‌تواند آن‌ها را به‌موقع تحلیل کند. MapReduce کمک می‌کند این حجم پراکنده و سنگین داده بین چندین ماشین تقسیم شود و هر بخش جداگانه پردازش گردد، به‌طوری‌که بتوان از دل این لاگ‌های عظیم الگوها، خطاها یا آمارها را استخراج کرد. فایل ورودی `access-2019-01-23.log` شامل مجموعه‌ای از لاگ‌های واقعی دسترسی به یک سایت فروشگاه ایرانی است که شما باید با استفاده از MapReduce به استخراج داده از آن بپردازید.

نمونه log :

```
54.36.149.41 - - [22/Jan/2019:03:56:14 +0330] "GET /filter/27|13%20... HTTP/1.1" 200 30577 "-"
  ↳ "Mozilla/5.0 ..." "-"
31.56.96.51 - - [22/Jan/2019:03:56:16 +0330] "GET /image/60844/... HTTP/1.1" 200 5667
  ↳ "https://www.zanbil.ir/m/filter/b113" "Mozilla/5.0 (...)" "-"
```

هر خط شامل فیلدهای زیر است:

IP: آدرس مبدا درخواست

identity: حساب کاربری معمولاً استفاده نمی‌شود (تقریباً همیشه "-")

user: اگر auth باشد، نام کاربر

timestamp: زمان دقیق درخواست

Request Line: درخواست HTTP

status: نتیجه درخواست

response size: حجم پاسخ

referrer: صفحه قبلی

user-agent: اطلاعات دستگاه، مرورگر، bot

برای انجام فرآیند MapReduce اسکریپت‌های map و reduce خود را به زبان پایتون نوشته و بر روی کلاستر hadoop خود deploy کنید:

- اسکریپت mapper: هر خط ورودی (sys.stdin) را می‌خواند، اطلاعات مورد نیاز را استخراج می‌کند (می‌توانید از regex و یا split استفاده کنید) سپس جفت کلید-مقدار (key-value) را پرینت می‌کند.
- اسکریپت reducer: ورودی مرتب‌شده را دریافت می‌کند، مقدارهای مربوط به هر کلید را جمع یا تجمیع می‌کند و نتیجه نهایی هر کلید را پرینت می‌کند.

تسک ۲.۱: یافتن تعداد درخواست‌های IP

در این بخش باید با استفاده از MapReduce تعداد درخواست‌های هر IP را محاسبه کنید. نمونه خروجی:

```
81.90.150.186    2
81.90.152.22    69
81.91.136.100   1
82.145.221.99   15
```

تسک ۲.۲: حجم ترافیک در هر ساعت از شبانه روز

در این بخش شما باید مجموع حجم ترافیک استفاده شده (فیلد Response Size) به ازای هر ساعت شبانه روز را بدست آورید. نمونه خروجی:

```
00    1830299
01    1429920
02    982299
03    2100050
04    890123
...
```

تسک ۲.۳: میانگین ساعتی ترافیک مصرفی (امتیازی)

میانگین ترافیک مصرفی در هر ساعت از شبانه روز را با MapReduce بدست آورید. از آنجایی که فایل access-2019-01-23.log تنها شامل log یک روز خاص می باشد، برای انجام این بخش باید از فایل access-long.log به عنوان ورودی استفاده کنید.

اجرای Hadoop

نکته: Hadoop معمولاً انتظار دارد که برنامه های MapReduce با زبان Java نوشته شوند، زیرا خود چارچوب Hadoop روی JVM اجرا می شود و کدهای کاربر را به صورت فایل های JAR بارگذاری می کند. اما برای استفاده از زبان های دیگر مانند Python، Hadoop رابطی به نام Hadoop Streaming ارائه می دهد که اجازه می دهد برنامه های خارجی به عنوان Mapper و Reducer استفاده شوند. در این روش، Hadoop هر رکورد ورودی را از طریق STDIN به برنامه mapper می فرستد و خروجی آن را از STDOUT می خواند؛ برای reducer نیز همین روند انجام می شود. به این ترتیب، هر زبانی که بتواند از ورودی استاندارد بخواند و روی خروجی استاندارد بنویسد (مانند پایتون) می تواند برای MapReduce استفاده شود.

برای اجرای فرآیند MapReduce می توانید از دستور زیر استفاده کنید:

```
hadoop jar /opt/hadoop-3.2.1/share/hadoop/tools/lib/hadoop-streaming-3.2.1.jar \  
-files mapper.py, reducer.py \  
-mapper mapper.py \  
-reducer reducer.py \  
-input access-2019-01-23.log \  
-output output
```

موارد زیر را به گزارش خود اضافه کنید:

- توضیح کامل کدهای mapper و reducer هر بخش
- نمایش و توضیح خروجی تولید شده توسط MapReduce هر بخش

فاز دوم: آشنایی با Spark

مقدمه

این تمرین بر تسلط به ویژگی‌ها و قابلیت‌های اساسی Apache Spark متمرکز است. شما مفاهیم اصلی مورد نیاز برای کار مؤثر با Spark در محیط‌های واقعی را خواهید آموخت.

برای تولید داده‌ها کد `generate_date.py` را اجرا کنید. برای انجام این تمرین بهتر است ابتدا اعداد کد تولید داده را تغییر دهید تا داده‌های کمتری تولید شود و در انتها با اعداد اصلی اجرا کنید. برای مثال:

```
n_devices = 1000          #Reduced from 5,000
n_readings = 1000000      #Reduced from 20,000,000
```

بهتر است این تمرین را در یک فایل `jupyter notebook` پیاده‌سازی کنید.

توضیح دیتاست

شما با داده‌های حسگرهای دستگاه‌های IoT کار خواهید کرد:

`sensor_readings.csv` :

```
device_id, timestamp, temperature, humidity, pressure, battery_level, status_code
```

`devices.csv` :

```
device_id, device_type, manufacturer, installation_date, max_temp, min_temp
```

Schema و مدیریت SparkSession

تسک ۱.۱: پیکربندی SparkSession

`SparkSession` نقطه ورود به تمام قابلیت‌های `Spark` است. مانند باز کردن یک اتصال به کلاستر `Spark`. از طریق `SparkSession` می‌توانید رفتار `Spark`، میزان حافظه مصرفی و بهینه‌سازی محاسبات را تنظیم کنید.

پیکربندی مهم است زیرا پیکربندی صحیح می‌تواند تسک‌های `Spark` شما را تا ۱۰ برابر سریع‌تر کند یا حتی مانع اجرای آن‌ها شود. بارهای کاری مختلف به تنظیمات متفاوتی نیاز دارند. مثلاً برنامه‌ای که پر از `Join` است با برنامه‌ای که پر از `Aggregation` است تنظیمات متفاوتی می‌خواهد.

یک `SparkSession` با پیکربندی مناسب ایجاد کنید:

- Application name
- Shuffle partitions: 200 (Controls parallelism for shuffles)
- Enable adaptive query execution (Lets Spark adjust plans at runtime)
- Broadcast join threshold: 10MB (Auto-broadcast tables smaller than this)
- Enable Kryo serialization (Faster than default Java serialization)

نیازمندی‌ها:

۱. ایجاد Spark Session با تنظیمات بالا

۲. چاپ تمام تنظیمات فعال

۳. نمایش اطلاعات کلاستر (executors، حافظه، سطح موازی‌سازی)

۴. نمایش یک مثال Accumulator (مثلاً شمارش رکوردهای خطا دار)

Accumulator چیست؟ Accumulator ها مانند شمارنده‌های سراسری در محاسبات توزیع شده هستند. به شما اجازه می‌دهند تا به صورت ایمن در تمام نودهای worker شمارش کنید (مثلاً تعداد خطاها یا رکوردهای پردازش شده). تحویلی: کد + خروجی تنظیمات

تسک ۲.۱: تعریف Schema و بارگذاری داده

Schema ساختار داده را تعریف می‌کند. تعریف می‌کند که کدام ستون‌ها وجود دارند و هر ستون چه نوعی دارد (date, integer, string و غیره). مانند یک قالب برای DataFrame است.

- Inferred Schema: Spark reads your data and guesses the types. Convenient but slow (reads data twice) and can guess wrong.
- Explicit Schema: You tell Spark exactly what types to expect. Faster (reads once) and prevents type errors.

داده‌های sensor_readings را با سه روش بارگذاری کنید:

۱. Inferred Schema با استفاده از inferSchema=True سپس schema را نمایش دهید و زمان را اندازه گیری کنید. در این روش spark داده را اسکن کرده و نوع ستون‌ها را حدس می‌زند.

- مزایا: آسان، بدون کار دستی

- معایب: کند (دو بار اسکن)، ممکن است نوع را اشتباه تشخیص دهد

۲. Explicit Schema با استفاده از StructType/StructField سپس schema را نمایش دهید و زمان را اندازه گیری کنید. در این روش شما نوع ستون‌ها را تعیین می‌کنید.

- مزایا: سریع (یک اسکن)، ایمن، قابل پیش‌بینی

- معایب: باید ساختار داده را از قبل بدانید

نیازمندی‌ها:

- مقایسه زمان بارگذاری (Inferred در مقابل Explicit)
- مدیریت رکوردهای خراب (با حالت PERMISSIVE و گرفتن رکوردهای بد)
- تحویلی: کد همراه با مقایسه زمان به همراه توضیح کوتاه برای هر بخش

حالت PERMISSIVE چیست؟

وقتی Spark با رکورد خراب مواجه می‌شود:

- ستون‌های سالم را نگه می‌دارد

- کل ردیف خراب را در یک ستون ویژه به نام corrupt_record می‌گذارد

- اجازه می‌دهد تسک ادامه پیدا کند به جای crash

بخش دوم: تبدیل‌های DataFrame و UDF ها

تسک ۱.۲: عملیات پایه DataFrame

DataFrame جدول‌های توزیع‌شده هستند. در این نوع جدول داده شما در چندین ماشین تقسیم شده اما مانند یک جدول واحد با آن کار می‌کنید. شبیه Pandas DataFrame است اما برای Big Data طراحی شده. Transformation ها عملیات‌هایی هستند که دیتافریم‌های جدیدی تولید می‌کنند (مانند نتایج فیلترشده، select شده و یا join ها) Transformation عملیات lazy هستند و تا زمانی که نتیجه درخواست نشود اجرا نمی‌شوند. این اجازه می‌دهد Spark کل pipeline را بهینه کند. عملیات‌های زیر را پیاده‌سازی کنید و زمان‌ها را اندازه‌گیری کنید:

۱. عملیات ستونی:

- افزودن ستون محاسبه‌شده:

`temp_fahrenheit = (temperature 9/5) + 32`

- ایجاد flag وضعیت:

`”CRITICAL” if temp > max_temp or temp < min_temp ”NORMAL” otherwise (use when().otherwise())`

- تجزیه timestamp برای استخراج ساعت و روز هفته.
- مدیریت مقادیر null در temperature (جایگزینی با میانگین)

۲. عملیات پیچیده:

- broadcast join بین دو فایل داده (عملیات join دو جدول را به کمک یک کلید ترکیب می‌کند). broadcast join نیز جدول را به همه worker ها ارسال می‌کند که از shuffling سریع‌تر است.
- GroupBy بر اساس device_type و محاسبه میانگین دما و تعداد خوانش‌ها. (group_by همانند این ویژگی در sql می‌باشد).
- استفاده از Window Function برای محاسبه میانگین متحرک ۳ خوانش برای هر دستگاه (مرتبه‌شده بر اساس timestamp). توابع پنجره‌ای روی یک «پنجره» از ردیف‌ها محاسبه می‌کنند (مثلاً ردیف فعلی + ۲ ردیف قبلی) که برای جمع، میانگین‌های متحرک و رتبه‌بندی‌ها مفید است.
- شناسایی ۵ دستگاه برتر از نظر تعداد خوانش در هر device_type (با رتبه‌بندی) (تابع row_number() رتبه‌ها را در هر گروه اختصاص می‌دهد).

تسک ۲.۲: توابع تعریف‌شده توسط کاربر (UDF)

هدف این بخش مقایسه عملکرد بین توابع داخلی و Pandas UDF و UDF است. UDF چیست؟ به شما اجازه می‌دهد منطق سفارشی را روی داده اعمال کنید که وقتی توابع داخلی Spark کافی نیستند استفاده می‌شود. چرا UDF کند است؟

- داده باید برای مسیر `JVM → Python → JVM` سریال‌سازی شود.
- توسط موتور Catalyst بهینه نمی‌شود.
- یک قانون طلایی این است که همیشه توابع داخلی را ترجیح دهید.

۱. یک UDF معمولی:

- طبقه‌بندی سطح باتری (با این کار ما یک منطق را پیاده‌سازی می‌کنیم):

`”LOW” (<20), ”MEDIUM” (20-60), ”HIGH” (>60)`

- تشخیص ناهنجاری دما برای وقتی که دما خارج از بازه `[min_temp, max_temp]` باشد که در صورتی که دما بیرون از بازه باشد true برمی‌گرداند.
- ثبت UDF برای استفاده در SQL. با این کار بعد از ثبت می‌توانید udf خود را در sql queries استفاده کنید.

- زمان این عملیات ها را وقتی از udf استفاده میکنیم و نمیکنیم را اندازه گیری کنید.

۲. Pandas UDF:

Pandas UDF: توابع وکتوریزه که کل ستون را یکجا به کمک Pandas/NumPy پردازش می کنند. به کمک Apache Arrow می توانند داده ها را سریع انتقال دهند (۵ تا ۱۰ برابر سریع تر از UDF معمولی).
توجه داشته باشید این قسمت به نسخه سازگار PyArrow نیاز دارد و ممکن است روی همه سیستم ها کار نکند. اگر UDF های Pandas را نتوانستید بالا بیاورید از این قسمت صرف نظر کنید. این بخش برای اهداف یادگیری امتیازی است.

- همان طبقه بندی باتری و دما را به صورت Pandas UDF پیاده سازی کنید.
- مقایسه عملکرد بین UDF معمولی و Pandas UDF و توابع داخلی.

۳. تحلیل:

- نمایش عملیات UDF
- توضیح اینکه هزینه سریال سازی چیست.
- توضیح اینکه زمان مناسب برای استفاده یا اجتناب از UDF چیست.

بخش سوم: Persistence و Caching

تسک ۱.۳: مقایسه سطوح ذخیره سازی

سه سطح ذخیره سازی را مقایسه کنید:

1. MEMORY_ONLY
2. MEMORY_AND_DISK
3. MEMORY_ONLY_SER

تسک ۲.۳: چه زمانی باید Cache کرد؟

نمایش دو سناریو: مفید بودن caching وقتی DataFrame چند بار استفاده می شود مانند فیلتر برای count و aggregation و سر بار caching وقتی فقط یک بار استفاده می شود.
تحویلی: توضیحات به اضافه کد یک مثال برای مقایسه که چه زمانی کش کردن مناسب و چه زمانی نامناسب است به همراه نتایج.

بخش چهارم: Join ها، Shuffle و بهینه سازی

تسک ۱.۴: استراتژی های Join

پیاده سازی و تحلیل Join های مختلف بین sensor readings و devices.

۱. انواع join های زیر اعمال کنید.

- (a) Inner join (default)
- (b) Left outer join (keep all readings even if device not found)

(c) Left anti join (find readings with no matching device)

۲. از استراتژی‌های زیر استفاده کنید:

- اجازه انتخاب خودکار به Spark
- اجبار به broadcast join

توضیح دهید که هر استراتژی کجا مناسب است. از تابع explain برای توضیح هر join استفاده کنید.
تحویلی: کد به همراه توضیحات

تسک ۲.۴: چالش بهینه‌سازی عملکرد (امتیازی)

کد ناکارآمد زیر را بهینه کنید و بهبود را اندازه‌گیری کنید:

```
readings = spark.read.csv("sensor_readings.csv", inferSchema=True)
devices = spark.read.csv("devices.csv", inferSchema=True)
readings.count()
result = readings.filter(col("temperature") > 30) \
    .join(devices, "device_id") \
    .groupBy("device_type").avg("temperature") \
    .join(devices, "device_type")
result.count()
result.show()
```

بهینه‌سازی شما باید شامل موارد زیر باشد:

- Explicit schema
- Broadcast join
- Caching where appropriate
- Remove redundant operations

تحویلی: اندازه‌گیری بهبود (زمان اجرا) و توضیح هر بهینه‌سازی